

Ağ Trafik Analizi ve Araçları: Kapsamlı Bilgilendirme Notu

Yönetici Özeti

Bu bilgilendirme notu, Ağ Trafik Analizi'nin (NTA) temel prensiplerini, bu analizde kullanılan temel araçları ve siber güvenlik olaylarını tespit etmeye yönelik pratik analiz tekniklerini kapsamlı bir şekilde özetlemektedir. NTA, yalnızca Wireshark gibi araçları kullanmaktan ibaret olmayıp, ağdaki veri akışını tam görünürlükle anlamak için log korelasyonu, derin paket denetimi ve ağ akış istatistiklerinin birleşimini içeren bir süreçtir. Temel amaçları arasında ağ performansını izlemek, anormallikleri tespit etmek ve siber saldırıları ortaya çıkarmak, doğrulamak ve yeniden yapılandırmak yer alır.

Analiz, TCP/IP yığınının katmanlarında (Uygulama, Taşıma, İnternet) gerçekleştirilir ve her katmandaki başlık bilgileri ile uygulama verileri (payload) incelenir. Bu inceleme, oturum kaçırma (session hijacking) ve parçalama saldırıları (fragmentation attacks) gibi gelişmiş tehditlerin tespitini mümkün kılar. Trafik akışları, genellikle LAN dışına çıkan/giren **Kuzey-Güney (North-South)** ve LAN içinde kalan **Doğu-Batı (East-West)** olarak sınıflandırılır; özellikle Doğu-Batı trafiğinin izlenmesi, ağ içinde yanal hareket eden saldırıların tespit etmek için kritiktir.

Wireshark ve **NetworkMiner** gibi araçlar bu analiz sürecinin merkezinde yer alır. Wireshark, derinlemesine paket analizi, güçlü filtreleme yetenekleri ve 3000'den fazla protokol desteği ile öne çıkarken; NetworkMiner, hızlı bir genel bakış, işletim sistemi tespiti (OS fingerprinting) ve dosya çıkarma gibi özellikleriyle "kolay hedefleri" (low-hanging fruits) toplamak için idealdir.

Pratik analiz senaryoları arasında, Nmap taramalarını (TCP Connect, SYN, UDP) pencere boyutu (window size) ve ICMP yanıtları gibi ayırt edici özelliklere dayanarak tespit etmek; ARP zehirlenmesi (MITM) saldırılarını yinelenen IP uyarıları ve yoğun ARP cevap paketleri aracılığıyla belirlemek; ve DHCP, NetBIOS ve Kerberos protokollerini analiz ederek ağdaki cihazların isimlerini (hostname) ve kullanıcı kimliklerini (username) ortaya çıkarmak bulunur. Ayrıca, verilerin ICMP ve DNS gibi güvenilir protokoller içine gizlenerek sızdırıldığı tünelleme saldırıları, anormal paket boyutları ve uzun sorgu isimleri ile tespit edilebilir. FTP ve HTTP gibi şifresiz protokollerin analizi, kullanıcı adı/şifre bilgilerinin çalınmasını, kaba kuvvet saldırılarını ve Log4j gibi web zafiyetlerini ortaya çıkarabilir. Şifreli HTTPS trafiği ise, doğru anahtar log dosyaları (SSLKEYLOGFILE) kullanılarak Wireshark üzerinde çözülebilir ve şeffaf hale getirilebilir.

Bölüm 1: Ağ Trafik Analizinin (NTA) Temelleri

NTA'nın Tanımı ve Kapsamı

Ağ Trafik Analizi (NTA), bir ağdaki veri akışını yakalama, inceleme ve analiz etme sürecidir. Temel hedefi, ağ içinde ve dışında neyin iletişim kurduğunu anlamak için tam bir görünürlük sağlamaktır. NTA, sadece Wireshark gibi bir araçtan ibaret değildir; daha geniş bir perspektifle, belirli hedefler doğrultusunda çeşitli logların ilişkilendirilmesi, derin paket denetimi (deep packet inspection) ve ağ akış istatistiklerinin bir kombinasyonudur.

Ağ trafiğini analiz etme becerisi, SOC L1 analistleri başta olmak üzere, mavi ve kırmızı takımlardaki birçok rol için temel bir yetenektir. L1 analistlerinin, büyük miktardaki ağ bilgisi arasında gezinebilmesi, normal olanı (baseline) anlaması ve normalden sapan durumları tespit edebilmesi beklenir.

Neden Ağ Trafiği Analiz Edilmelidir?

Ağ trafiği analizi genel olarak şu amaçlarla kullanılır:

- **Ağ Performansını İzleme:** Ağdaki ani performans artışları veya yavaşlamalar gibi anormallikleri kontrol etmek.
- **Anormallikleri Kontrol Etme:** Normal dışı aktiviteleri tespit etmek.
- **Şüpheli İletişim İçeriğini İnceleme:** İç ve dış ağdaki şüpheli iletişimlerin içeriğini denetlemek. Örneğin, DNS üzerinden veri sızdırma, HTTP üzerinden zararlı bir ZIP dosyasının indirilmesi veya ağ içinde yanal hareketler.

Bir Güvenlik Operasyonları Merkezi (SOC) perspektifinden bakıldığında, NTA şu konularda yardımcı olur:

- Şüpheli veya kötü amaçlı faaliyetlerin tespiti.
- Olay müdahalesi sırasında saldırıların yeniden yapılandırılması.
- Oluşturulan uyarıların (alerts) doğrulanması ve onaylanması.

Gözlemlenebilir Ağ Trafiği: TCP/IP Katmanları

Ağda gözlemlenebilecek trafik, neredeyse her ağ arayüzüne sahip cihazda uygulanan **TCP/IP yığını** mimarisi kullanılarak en iyi şekilde açıklanabilir. Her katman, veriyi bir sonraki katmana iletmek için gereken bilgileri (başlıklar) tanımlar. Bu başlıklarda yer alan bilgiler ve uygulama verileri, analiz edilmek istenen temel unsurlardır.

- **Uygulama (Application) Katmanı:** Bu katmanda uygulama başlık bilgileri ve asıl veri olan **payload** bulunur. Bu bilgiler, kullanılan uygulama katmanı protokolüne göre değişiklik gösterir.
- **Taşıma (Transport) Katmanı:** Uygulama verisi ve başlığı, taşıma katmanında daha küçük parçalara bölünür (segmentasyon) ve kapsüllenir. Her parçaya, çoğunlukla TCP veya UDP olan bir taşıma başlığı eklenir. Güvenlik duvarı logları genellikle kaynak ve hedef portları ile bayrakları (flags) içerir, ancak diğer alanlar genellikle dahil edilmez. Bu eksik alanlar, **Oturum Kaçırma (Session Hijacking)** gibi

saldırıların tespitinde kritik olabilir. Bu tür saldırılar, başlıktaki **sıra numaraları (sequence numbers)** analiz edilerek tespit edilebilir.

- **İnternet Katmanı:** Taşıma katmanından gelen bir segment, internet katmanında kendi başlığını alır. Eğer segment, **Maksimum İletim Birimi'nden (MTU)** büyükse, **parçalara (fragments)** bölünür. Bu katmanda en sık loglanan alanlar kaynak ve hedef IP ile TTL'dir. Ancak, parçalama (fragmentation) saldırılarını tespit etmek için "fragment offset" ve "total length" gibi alanların incelenmesi gerekir. Saldırganlar, IDS'leri atlatmak için küçük parçalar oluşturabilir veya **örtüşen (overlapping)** byte aralıkları kullanarak parçaların yeniden birleştirilmesini bozabilirler. Örneğin, bir paketin offset değeri, önceki bir paketin kapladığı byte aralığı ile çakışırsa, bu bir "overlap" durumudur ve IDS'leri atlatmak için kullanılabilir.

Ağ Trafiği Kaynakları ve Akışları

Ağ trafiği kaynakları ve akışları, kurumsal bir ağdaki önceden belirlenmiş yapılara göre gruplandırılabilir.

Kaynaklar

Trafik kaynakları iki ana kategoriye ayrılır: ara cihazlar ve uç nokta cihazları.

- **Ara Cihaz Kaynakları (Intermediary Sources):** Trafiğin çoğunlukla üzerinden geçtiği cihazlardır. Bu kategoriye **güvenlik duvarları, anahtarlar, web proxy'leri, IDS, IPS, yönlendiriciler, erişim noktaları ve kablosuz LAN denetleyicileri** girer. Bu cihazlar tarafından üretilen trafik, yönlendirme protokolleri (EIGRP, OSPF), yönetim protokolleri (SNMP, PING) ve destekleyici protokollerden (ARP, DHCP) kaynaklanır.
- **Uç Nokta Kaynakları (Endpoint Sources):** Trafiğin başladığı ve bittiği cihazlardır ve ağ bant genişliğinin büyük kısmını kullanırlar. Bu kategoriye **sunucular, istemciler, IoT cihazları, yazıcılar, sanal makineler, bulut kaynakları, mobil telefonlar ve tabletler** dahildir.

Akışlar

Ağ trafiği akışları, ağdaki hizmetlere göre belirlenir ve iki ana gruba ayrılır:

- **Kuzey-Güney Trafiği (North-South Traffic):** LAN'dan WAN'a çıkan veya WAN'dan LAN'a giren trafiktir. Bu trafik genellikle güvenlik duvarından geçtiği için yakından izlenir. HTTPS, DNS, SSH, VPN ve RDP gibi istemci-sunucu protokolleri bu kategoriye girer. **Güvenlik duvarı kurallarının ve loglamanın doğru yapılandırılması, bu alanda görünürlük için anahtardır.**
- **Doğu-Batı Trafiği (East-West Traffic):** Kurumsal LAN içinde kalan trafiktir ve genellikle daha az izlenir. Ancak, bir ağ ele geçirildiğinde, saldırgan genellikle **ağ içinde yanal olarak hareket etmek (lateral movement)** için bu kategorideki hizmetleri (Active Directory, dosya paylaşımları, uygulama iletişimi vb.) kullanır. Bu nedenle bu akışların takibi hayati önem taşır.

Bilgi Toplama Yöntemleri

Ağ trafiği analizi için bilgi üç ana yolla elde edilir:

1. **Loglar:** Ağdaki her sistem ve protokolün bir loglama yöntemi vardır. Ancak evrensel bir standart yoktur; her üretici kendi loglama formatını (örn: Windows Event Logs, Syslog, CLF) ve içeriğini belirler. Loglar genellikle tam paket detaylarını içermez, sadece kaynak IP ve hedef IP gibi faydalı görülen alanları kaydeder.
2. **Tam Paket Yakalama (Full Packet Capture):** Loglar yetersiz kaldığında, tam paketlerin yakalanması ve incelenmesi gerekir. Bu iki yöntemle yapılır:
 - **Ağ TAP (Network Tap):** Ağ fiziksel olarak yerleştirilen bir cihazdır. Performansı etkilemeden geçen tüm trafiğin bir kopyasını oluşturur ve bu kopyayı analiz için özel bir izleme portuna yönlendirir.
 - **Port Yansıtma (Port Mirroring):** Bir ara cihaz üzerindeki bir porttan geçen paketlerin, bir başka porta yazılımsal olarak kopyalanmasıdır. Cisco bu özelliği **SPAN** olarak adlandırır.
3. **Ağ İstatistikleri:** Ağdaki genel eğilimleri ve anormallikleri anlamak için kullanılır.

Bölüm 2: Ağ Analiz Araçları: Wireshark ve NetworkMiner

Wireshark

Wireshark, canlı trafiği yakalayabilen (sniffing) ve paket yakalama dosyalarını (PCAP) inceleyebilen, açık kaynaklı ve platformlar arası bir ağ paketi analiz aracıdır. Alanındaki en iyi araçlardan biri olarak kabul edilir.

Temel Özellikler ve Kullanım

- **Dosya Detaylarını Görüntüleme:** Özellikle birden fazla pcap dosyasıyla çalışırken, dosyayı tanımlamak ve önceliklendirmek için dosya karması (hash), yakalama zamanı ve arayüz gibi ayrıntıları bilmek önemlidir. Bu bilgilere **"Statistics --> Capture File Properties"** menüsünden erişilebilir.
- **Pakete Gitme (Go to Packet):** Belirli paket numaralarına hızla atlamayı sağlar. Bu özellik, yalnızca paketler arasında gezinmekle kalmaz, aynı zamanda bir konuşmanın belirli bir bölümündeki sonraki paketi bulmaya da yardımcı olur.
- **Paket Bulma (Find Packet):** Paket içeriğine göre arama yapma imkanı sunar. **"Edit --> Find Packet"** menüsü kullanılarak dört farklı girdi türüyle (Display filter, Hex, String, Regex) arama yapılabilir. Arama yapılacak bölmenin (paket listesi, detaylar, byte'lar) doğru seçilmesi kritiktir.
- **Zaman Görüntüleme Formatı:** Varsayılan olarak zamanı "Yakalamanın Başlangıcından İtibaren Saniyeler" olarak gösterir. Daha iyi bir görünüm için genellikle UTC formatı tercih edilir. Bu ayar **"View --> Time Display Format"** menüsünden değiştirilebilir.

İleri Düzey Operasyonlar ve Filtreleme

- **İstatistikler (Statistics):** Bu menü, trafik kapsamı, mevcut protokoller, uç noktalar ve konuşmalar hakkında hızlı bir özet sunarak analistlerin bir hipotez oluşturmalarına yardımcı olan çok sayıda istatistiksel seçenek sunar.
- **Görüntüleme Filtresi Sözdizimi (Display Filter Syntax):** Wireshark'ın en güçlü özelliğidir. 3000'den fazla protokolü destekler ve paket düzeyinde arama yapılmasına olanak tanır.
 - **Karşılaştırma Operatörleri:** `==` (eşit), `!=` (eşit değil), `>` (büyük), `<` (küçük) gibi operatörler kullanılarak filtreler oluşturulabilir. Filtrelerde ondalık ve onaltılık değerler desteklenir.
 - **Mantıksal İfadeler:** `and` (`&&`), `or` (`||`), `not` (`!`) gibi boolean operatörleri ile karmaşık filtreler oluşturulabilir.

Operatör (İngilizce)	Operatör (C-Like)	Açıklama	Örnek
<code>eq</code>	<code>==</code>	Eşit	<code>ip.src == 10.10.10.100</code>
<code>ne</code>	<code>!=</code>	Eşit değil	<code>ip.src != 10.10.10.100</code>
<code>gt</code>	<code>></code>	Büyüktür	<code>ip.ttl > 250</code>
<code>lt</code>	<code><</code>	Küçüktür	<code>ip.ttl < 10</code>
<code>ge</code>	<code>>=</code>	Büyük veya eşit	<code>ip.ttl >= 0xFA</code>
<code>le</code>	<code><=</code>	Küçük veya eşit	<code>ip.ttl <= 0xA</code>
<code>and</code>	<code>&&</code>	Mantıksal VE	<code>(ip.src == 10.10.10.100) and (ip.src == 10.10.10.111)</code>
<code>or</code>	<code> </code>	Mantıksal VEYA	<code>(ip.src == 10.10.10.100) or (ip.src == 10.10.10.111)</code>
<code>not</code>	<code>!</code>	Mantıksal DEĞİL	<code>!(ip.src == 10.10.10.222)</code>

- **Filtre Araç Çubuğu:** Filtrelerin oluşturulduğu ve uygulandığı akıllı bir araç çubuğudur. Filtrelerin geçerliliğini renklerle belirtir: **Yeşil** (geçerli), **Kırmızı** (geçersiz), **Sarı** (uyarı, çalışır ancak güvenilir değildir).
- **Özel Filtreler:**
 - **`contains`:** Paketler içinde belirli bir değeri (büyük/küçük harfe duyarlı) arar. Örnek: `http.server contains "Apache"`

- **matches**: Bir düzenli ifade (regex) kalıbını (büyük/küçük harfe duyarsız) arar. Örnek: `http.host matches "\.(php|html)"`
- **in**: Bir değeri veya alanı belirli bir kapsam/aralık içinde arar. Örnek: `tcp.port in {80 443 8080}`

NetworkMiner

NetworkMiner, açık kaynaklı bir trafik yakalama, PCAP işleme ve protokol analiz aracıdır.

Çalışma Modları ve Özellikler

- **Yakalam Modu (Sniffer Mode)**: Sadece Windows'ta kullanılabilen bir trafik yakalama özelliğine sahiptir. Ancak bu özellik, aracın ana amacı değildir ve diğer özellikleri kadar güvenilir bulunmamaktadır.
- **Paket Ayrıştırma/İşleme (Packet Parsing/Processing)**: Aracın asıl gücü, yakalanan trafik dosyalarını (PCAP) ayrıştırarak hızlı bir genel bakış ve bilgi sunmasıdır. Bu mod, derinlemesine bir araştırmadan önce "kolay hedefleri" toplamak için önerilir.

Wireshark ile Karşılaştırma

Özellik	NetworkMiner	Wireshark
Amaç	Hızlı genel bakış, trafik haritalama ve veri çıkarma	Derinlemesine analiz
GUI	✓	✓
Trafik Yakalama	✓	✓
PCAP İşleme	✓	✓
OS Tespiti	✓	✗
Parametre/Anahtar Kelime Keşfi	✓	Manuel
Kimlik Bilgisi Keşfi	✓	✓
Dosya Çıkarma	✓	✓
Filtreleme Seçenekleri	Sınırlı	✓
Paket Kod Çözme	Sınırlı	✓
Protokol Analizi	✗	✓

Payload Analizi	✗	✓
İstatistiksel Analiz	✗	✓
Platform Desteęi	✓	✓
Host Kategorizasyonu	✓	✗
Yönetim Kolaylığı	✓	✓

Bölüm 3: Wireshark ile Pratik Trafik Analizi ve Saldırı Tespiti

Ağ Keşfi ve Tarama Tespiti (Nmap)

1. TCP Connect Taramaları (nmap -sT)

- **Özellik:** 3'lü el sıkışma (3-way handshake) sürecini tamamlar.
- **Ayırt Edici Özellik:** **Pencere Boyutu (Window Size)** genellikle 1024 bayttan büyüktür.
- **Davranış:** Açık portlara **SYN**, **SYN/ACK**, **ACK** dizisiyle tam bağlantı kurar. Kapalı portlar **RST/ACK** ile yanıt verir.
- **Wireshark Filtresi:** `tcp.flags.syn == 1 and tcp.flags.ack == 0 and tcp.window_size > 1024`

2. SYN "Stealth" Taramaları (nmap -sS)

- **Özellik:** 3'lü el sıkışma sürecini tamamlamaz (yarı açık tarama).
- **Ayırt Edici Özellik:** **Pencere Boyutu (Window Size)** genellikle 1024 bayta eşit veya daha küçüktür.
- **Davranış:** Açık bir porttan **SYN/ACK** yanıtı aldığıında, bağlantıyı tamamlamak yerine **RST** göndererek hemen keser.
- **Wireshark Filtresi:** `tcp.flags.syn==1 and tcp.flags.ack==0 and tcp.window_size <= 1024`

3. UDP Taramaları (nmap -sU)

- **Özellik:** UDP bağlantısız olduğu için el sıkışma yoktur.
- **Davranış:** Açık portlar genellikle yanıt vermez. Kapalı bir porta yapılan istek ise hedef sunucudan **ICMP Type 3, Code 3 (Destination Unreachable: Port Unreachable)** hatası ile yanıtlanır.

- **Wireshark Filtresi:** `icmp.type==3 and icmp.code==3`

Yerel Ağ Saldırıları

ARP Zehirlenmesi (Man-in-the-Middle) Tespiti

- **Zafiyet:** ARP, "durumsuz" (stateless) bir protokoldür. Bir cihaz, talep etmemiş olsa bile kendisine gönderilen bir ARP Cevap (Reply) mesajına sorgusuz sualsiz güvenir.
- **Saldırı Mantığı:** Saldırgan, hem kurbanı hem de ağ geçidine (gateway) sürekli olarak kendisini diğer tarafmış gibi tanıtan sahte ARP Cevap paketleri gönderir ve trafiği kendi üzerine çeker.
- **Wireshark ile Tespit:**
 - **Filtre:** `arp` filtresi kullanılır. `arp.opcode == 2` ile sadece cevap paketleri incelenebilir.
 - **Gürültü/Spam:** Saldırgan, ARP tablosunu sürekli olarak zehirlemek için yoğun bir şekilde ARP Cevap paketleri gönderir. Trafikteki anormal ARP yoğunluğu bir işarettir.
 - **Wireshark Uyarısı:** Wireshark, aynı IP adresinin kısa süre içinde iki farklı MAC adresiyle ilişkilendirildiğini tespit ederse, **"Duplicate IP address detected"** uyarısı verir. Bu, bir ARP zehirlenmesi saldırısının güçlü bir kanıtıdır.

Kimlik ve Cihaz Tespiti

Hostname ve Kullanıcı Adı Tespiti

- **DHCP Analizi:** Cihazlar IP adresi talep ederken genellikle kendi isimlerini (Hostname) de pakete eklerler. `dhcp.option.dhcp == 3` filtresi ile **DHCP Request** paketleri incelenerek cihaz isimleri elde edilebilir.
- **NetBIOS (NBNS) Analizi:** Yerel ağdaki cihazların birbirleriyle isimleri üzerinden iletişim kurmasını sağlayan eski bir protokoldür. `nbns` filtresi ile yapılan sorgular incelenerek cihaz isimleri bulunabilir.
- **Kerberos Analizi:** Windows domain ortamlarında varsayılan kimlik doğrulama protokolüdür. Bu trafik, hem bilgisayar isimlerini hem de kullanıcı adlarını içerir.
 - **Bilgisayar İsimleri:** Genellikle sonu `$` işaretiyle biter (Örn: `DESKTOP-123$`).
 - **Kullanıcı Adları:** `$` işareti içermez (Örn: `samet.kocats`).
 - **Filtreler:** Genel arama için `kerberos`. Kullanıcı adı aramak için `kerberos.CNameString contains "keyword"`. Sadece kullanıcı adlarını bulmak için `kerberos.CNameString and !(kerberos.CNameString contains "$")`.

Veri Sızdırma ve Tünelleme Teknikleri

Tünelleme, veriyi başka bir protokolün içine gizleyerek taşıma işlemidir. Saldırganlar genellikle güvenlik duvarlarının izin verdiği ICMP ve DNS gibi protokolleri bu amaçla kullanır.

- **ICMP Tünelleme Tespiti:** Saldırganlar, normalde küçük boyutlu olan Ping (ICMP) paketlerinin veri taşıma alanına çaldıkları verileri gizlerler.
 - **Tespit Yöntemi:** Ağda anormal derecede **büyük boyutlu Ping paketleri** görmek şüphe uyandırmalıdır.
 - **Wireshark Filtresi:** `data.len > 64 and icmp`
- **DNS Tünelleme Tespiti:** Saldırgan, çaldığı veriyi DNS sorgularının subdomain kısmına kodlayarak kendi kontrolündeki DNS sunucusuna gönderir.
 - **Tespit Yöntemi:** Çok **uzun ve anlamsız karakterlerden oluşan subdomain sorguları** veya bir istemciden aynı domaine yönelik anormal sayıda sorgu yapılması tünelleme belirtisidir.
 - **Wireshark Filtreleri:** `dns.qry.name.len > 15 and !mdns` veya bilinen araçları aramak için `dns contains "dnscat"`.

Uygulama Katmanı Protokol Analizi

FTP Trafik Analizi

- **Risk:** FTP, şifreleme kullanmadığı için ağ trafiğini dinleyen bir saldırgan kullanıcı adlarını, şifreleri ve aktarılan dosyaları açık metin (cleartext) olarak görebilir.
- **Analiz ve Saldırı Tespiti:**
 - **Kimlik Bilgileri:** `ftp.request.command == "USER"` ve `ftp.request.command == "PASS"` filtreleri ile kullanıcı adı ve şifreler görülebilir.
 - **Başarılı Giriş:** `ftp.response.code == 230` filtresi, başarılı kullanıcı girişlerini gösterir.
 - **Başarısız Giriş:** `ftp.response.code == 530` filtresi, başarısız giriş denemelerini gösterir. Çok sayıda `530` kodu, bir **Kaba Kuvvet (Brute-Force)** saldırısına işaret edebilir.

HTTP Trafik Analizi

- **Önem:** HTTP şifresiz olduğu için veri sızdırma, web saldırıları ve C2 trafiği için saldırganların sıkça kullandığı bir alandır.
- **Analiz Noktaları:**
 - **İstek Metotları:** `http.request.method == "POST"` filtresi, sunucuya veri gönderilen (giriş yapma, dosya yükleme) işlemleri izlemek için kritiktir.
 - **Cevap Kodları:** `404` (Not Found) kodları tarama aktivitesine, `401/403` (Unauthorized/Forbidden) kodları yetkisiz erişim denemelerine, `500` (Internal Error) kodları ise bir zafiyetin tetiklenmiş olabileceğine işaret edebilir.
 - **User-Agent Analizi:** Saldırganlar genellikle User-Agent başlığını taklit eder. Aynı IP'den gelen farklı User-Agent'lar, yazım hataları veya `Nmap`, `Sq1map` gibi araçların varsayılan kimlikleri şüphe uyandırmalıdır. **Log4j** zafiyetinde olduğu gibi, saldırı kodu (`jndi:ldap`) User-Agent içine gizlenebilir.

HTTPS (TLS) Trafiğinin Şifresini Çözme

- **Sorun:** HTTPS, TLS protokolü ile trafiği şifrelediği için içerik doğrudan görülemez. Analist için bu bir "karanlık oda"dır.
- **Şifreyi Çözmeden Görülebilenler:** Şifreli oturum başlamadan önceki **El Sıkışma (Handshake)** aşaması şifresizdir. `tls.handshake.type == 1` (Client Hello) ve `tls.handshake.type == 2` (Server Hello) filtreleri ile en azından kimin kimle şifreli bağlantı kurmaya çalıştığı görülebilir.
- **Şifre Çözme Yöntemi:**
 1. **Anahtar Log Dosyası:** Tarayıcılar (Chrome, Firefox), `SSLKEYLOGFILE` ortam değişkeni ayarlandığında, oturum şifreleme anahtarlarını bir metin dosyasına kaydedebilir.
 2. **Kritik Kural:** Anahtar kaydı, trafik kaydı (pcap) ile **eş zamanlı** yapılmalıdır.
 3. **Wireshark'ta Uygulama:** "Edit --> Preferences --> Protocols --> TLS" menüsünden `(Pre)-Master-Secret log filename` kısmına anahtar log dosyası gösterilir. Bu işlemten sonra, daha önce "Application Data" olarak görünen şifreli trafik, okunabilir HTTP/HTTP2 paketlerine dönüşür.